535514 Spring 2025: Reinforcement Learning **(Due: 2026/05/25, 21:00)**

Homework 3: Value-Based RL and Soft Actor-Critic

Submission Guidelines: Your deliverables shall consist of 2 separate files – (i) A PDF file: Please compile all your write-ups into one .pdf file (photos/scanned copies are acceptable; please make sure that the electronic files are of good quality and reader-friendly); (ii) A zip file: Please compress all your source code (including **sac.py** and **sac_halfcheetah.py**) into one .zip file. Please submit your deliverables via E3.

Problem 1 (Soft Policy Improvement for Regularized MDPs) (20 points)

In this problem, let us verify the policy update of Soft Policy Iteration: In the k-th iteration, given the entropy-regularized Q function $Q_\Omega^{\pi_k}$ with $\Omega(\pi(\cdot|s)) := \sum_{a\in\mathcal{A}} \pi(a|s)\log\pi(a|s)$, under Soft Policy Iteration, the new policy for the $k+1$-iteration can be obtained by solving the following optimization problem for each state $s \in \mathcal{S}$:

$$\pi_{k+1}(\cdot|s) = \arg\max_{\pi} \big\{\langle \pi(\cdot|s), Q_\Omega^{\pi_k}(s,\cdot)\rangle - \Omega(\pi(\cdot|s))\big\}. \tag{1}$$

Note that we can further write the above optimization problem in a more explicit manner:

$$\max_{\pi(\cdot|s)} \quad \sum_{a\in\mathcal{A}} \big(\pi(a|s)Q_\Omega^{\pi_k}(s,a) - \pi(a|s)\log\pi(a|s)\big), \quad \text{subject to } \sum_{a\in\mathcal{A}} \pi(a|s) - 1 = 0, \tag{2}$$

where the constraint is meant to ensure that π is a valid policy. Please show that the optimal solution to the above optimization problem is

$$\pi_{k+1}(\cdot|s) = \frac{\exp(Q_\Omega^{\pi_k}(s,\cdot))}{\sum_{a\in\mathcal{A}}\exp(Q_\Omega^{\pi_k}(s,a))}. \tag{3}$$

(Hint: To show (3), we leverage the Lagrange multiplier technique that we learn in the Calculus class. Specifically, let $\mu \in \mathbb{R}$ be the *Lagrange multiplier* associated with the constraint in (2). Then, we can construct the *Lagrangian* as

$$L(\pi) := \sum_{a\in\mathcal{A}} \big(\pi(a|s)Q_\Omega^{\pi_k}(s,a) - \pi(a|s)\log\pi(a|s)\big) - \mu\Big(\sum_{a\in\mathcal{A}} \pi(a|s) - 1\Big). \tag{4}$$

Then, the optimal solution satisfies $\frac{\partial L(\pi)}{\partial \pi(a|s)} = 0$, for every $a \in \mathcal{A}$.)

Problem 2 (Soft Actor-Critic for Continuous Control) (10+10+10+25+25=80 points)

In this problem, we will take a deeper look at the actual implementation of Soft Actor-Critic (SAC) algorithm. Please first take a look at the attached file **sac.py** and answer the following questions and finish the implementation in **sac.py**. For ease of exposition, the pseudo code of SAC is provided as below.

(a) (**Actor Network of SAC**) As shown by the source code, the first major class is **Actor**. There are three salient designs that are not completely addressed in the pseudo code:

- The output layer of the actor network produces the mean and the logarithm of the standard deviation (Lines 45-51 and Lines 58-74 in **sac_starter.py**). Why is this needed?
- We mentioned the "reparamterization trick". Could you carefully explain how this trick is implemented in SAC? (Hint: Lines 67-68 in **sac_starter.py**)
- In Lines 72-74 of **sac_starter.py**, the calculation of log probability at the actor network enforces some additional manipulations. Why is this needed? (Hint: You may check Appendix C of the original SAC paper https://arxiv.org/abs/1801.01290)

Algorithm 1 Soft Actor Critic Algorithm

Initialize:
Policy network $\pi_\phi(a|s)$
Q-networks $Q_{\theta_1}(s,a)$, $Q_{\theta_2}(s,a)$
Value network $V_\psi(s)$ and target value network $V_{\bar{\psi}}(s)$ with $\bar{\psi} \leftarrow \psi$
Replay buffer $\mathcal{D}$
for each iteration **do**
  for each environment step **do**
    Sample action $a_t \sim \pi_\phi(\cdot|s_t)$
    Execute a_t, observe reward r_t, and next state s_{t+1}
    Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
  end for
  for each gradient step **do**
    Sample minibatch $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ from $\mathcal{D}$
    Update Q-networks:
    Compute target value:

$$y_i = r_i + \gamma V_{\bar{\psi}}(s'_i)$$

    Minimize loss:

$$\mathcal{L}_{Q_j} = \frac{1}{N}\sum_i \left(Q_{\theta_j}(s_i, a_i) - y_i\right)^2, \quad j = 1, 2$$

    Update value network:
    Sample $a_i \sim \pi_\phi(\cdot|s_i)$, compute $\log \pi_\phi(a_i|s_i)$
    Compute soft Q estimate:

$$\hat{Q}_i = \min(Q_{\theta_1}(s_i, a_i), Q_{\theta_2}(s_i, a_i))$$

    Minimize value loss:

$$\mathcal{L}_V = \frac{1}{N}\sum_i \left(V_\psi(s_i) - (\hat{Q}_i - \alpha \log \pi_\phi(a_i|s_i))\right)^2$$

    Update policy network:

$$\mathcal{L}_\pi = \frac{1}{N}\sum_i \left(\alpha \log \pi_\phi(a_i|s_i) - Q_{\theta_1}(s_i, a_i)\right)$$

    Soft update for target value network:

$$\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$$

  end for
end for

(b) (**Soft Q Network of SAC**) Another important class is **CriticQ**. There are also several important tricks integrated with SAC.

- It appears that SAC uses two soft Q networks for the critic (in both the pseudo code and the python code). Why is this needed? Could you explain what issue this "twin Q" manages to address? (Hint: This trick already appears in TD3)
- Based on the above, could you write down the exact mathematical expression of the loss function used for the update the two soft Q networks?

(c) (**Alpha Network of SAC**) The vanilla SAC presumes using a fixed temperature parameter α for the entropy bonus. Subsequently, the enhanced SAC in https://arxiv.org/abs/1812.05905 introduced an auto-tuning mechanism of α.

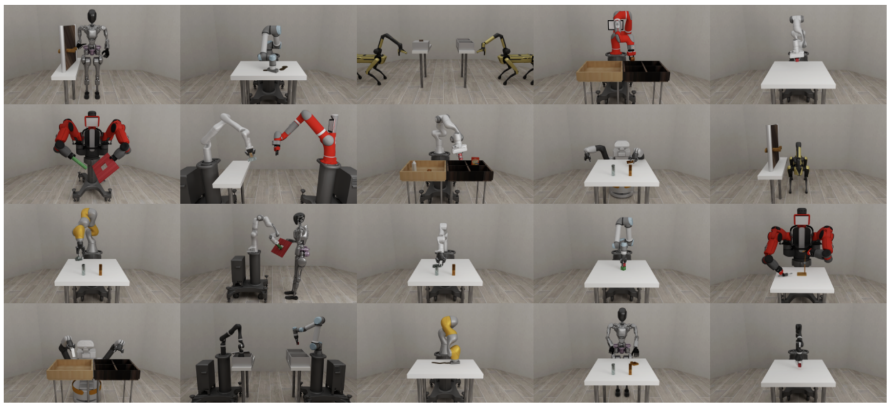

Figure 1: Example tasks in Robosuite.

- This auto-tuning scheme is implemented in Line 286 of **sac_starter .py**. Can you explain why this is implemented in this way? (Hint: You can refer to Section 5 of https://arxiv.org/abs/1812.05905)

(d) (**SAC on Pendulum-v1**) Next, let's finish the implementation of **sac_starter .py**. Specifically, you need to implement the following three things in PyTorch:

- Computing the Q loss in the **update_model** function (about 10 lines).
- Computing the V loss in the **update_model** function (no about 5 lines).
- Computing the actor loss in the **update_model** function (no about 5 lines).

We start by solving the simple "Pendulum-v1" problem (https://www.gymlibrary.dev/environments/classic_control/pendulum/) using the SAC algorithm. Please briefly summarize your results (including the snapshots of Weight & Bias record) in the report and document all the hyperparameters (e.g., learning rates, NN architecture, and batch size) of your experiments. The recommended package versions are as follows:

- `Python`: >= 3.8 (tested on 3.11)
- `gymnasium`: 1.1.1
- `torch`: >= 2.0 (tested on 2.6.0)
- `wandb`: >= 0.16 (tested on 0.19.10)

(Note: Pendulum-v1 is a rather basic environment mostly for the purpose of sanity check. As a result, typically it would take no more than 50,000 training steps to reach a well-performing policy of evaluation score above -170)

(e) (**SAC in Door Opening**) Based on your implementation for (d), please adapt your SAC algorithm to solve the "Panda-Door-Opening" robot arm manipulation task in Robosuite (Github: https://github.com/ARISE-Initiative/robosuite; Documentation: https://robosuite.ai/docs/overview.html; Paper: https://arxiv.org/abs/2009.12293). Save your code in another python file named **sac_panda_door.py**. Please add comments to your code whenever needed for better readability.

Train your Panda robot arm under SAC for 1 million environment steps and reach an evaluation score of at least 200 within 500k steps. Again, briefly summarize your results in the report and document all the hyperparameters of your experiments. The recommended package versions are as follows:

- `Python`: >= 3.8 (tested on 3.11)
- `gymnasium`: 1.1.1
- `torch`: >= 2.0 (tested on 2.6.0)
- `wandb`: >= 0.16 (tested on 0.19.10)
- `robosuite`: 1.5

(Note: As Door Opening is a more challenging environment than Pendulum, it would take a bit more training time to reach a well-performing policy, and it might require some efforts to tune the hyperparameters, e.g., learning rates and the batch sizes. That being said, it is typically expected that an SAC policy can attain an evaluation score of more than 400 in Door Opening in 500k training steps.)
For the deliverables, please submit the following:

- Technical report: Please summarize all your experimental results in 1 single report (and please be brief)
- All your source code
- Your well-trained models (both the actor and critic networks) saved in either .pth files or .ckpt files
- Record a short video (3-5 minute long) to describe your design and the experimental results that you observe in the following Problem 3(d)-3(e).